

Installation et configuration d'Android studio

Table des matières

Téléchargement et installation d' <i>Android Studio</i>	2
Votre première application Android	2
Création d'un projet	2
Utilisation de l'émulateur Android	9
Exercices	12
Références	13

Conçu et distribué par Google, *Android Studio* est l'environnement de développement intégré (IDE) officiel des applications destinées à la plate-forme Android. Un IDE propose des outils qui permettent aux développeurs de concevoir, compiler, exécuter et tester facilement des applications. La plate-forme *Android Studio* est basée sur *IntelliJ IDEA*. Elle inclut le greffon Android préinstallé ainsi que des modifications spécifiques à la plate-forme Android.

Dans ce chapitre, la création d'une application ainsi que l'utilisation de l'émulateur Android sont présentés.

Téléchargement et installation d'*Android Studio*

Vous référer à documentation propre à votre système d'exploitation: <https://developer.android.com/studio/install?hl=fr>

Votre première application Android

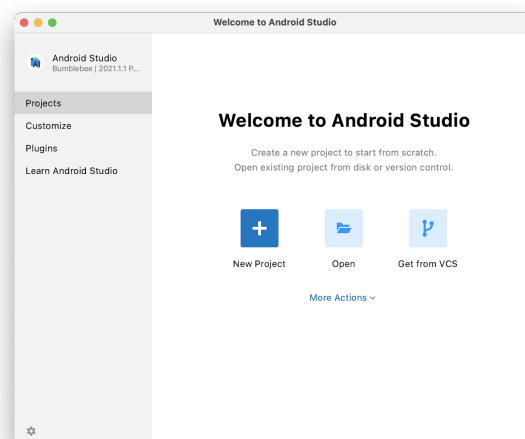
Création d'un projet

Pour créer un projet dans *Android Studio*:

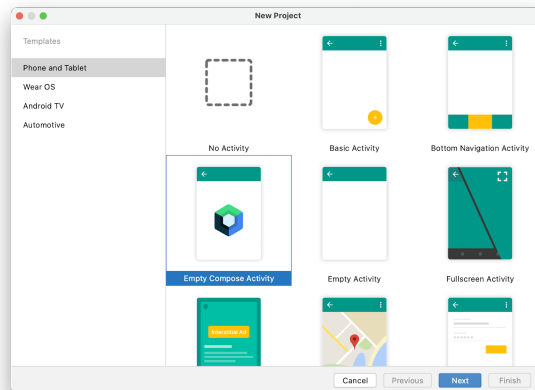
1. Double-cliquez sur l'icône **Android Studio** pour lancer *Android Studio*.



2. Dans la boîte de dialogue **Bienvenue dans Android Studio**, cliquez sur **Nouveau projet**.



La fenêtre **Nouveau projet** s'ouvre et affiche la liste des modèles fournis par Android Studio.



Dans *Android Studio*, un modèle de projet est un projet Android qui fournit le plan pour un certain type d'application. Les modèles créent la structure du projet et les fichiers dont Android Studio a besoin pour créer votre projet. Le code de démarrage mis à votre disposition pour une mise en route rapide dépend du modèle que vous choisissez.

3. Assurez-vous que l'onglet **Téléphone et tablette** est sélectionné.
4. Cliquez sur le modèle **Activité Compose vide** afin de le sélectionner comme modèle pour votre projet. Le **modèle "Activité Compose vide"** sert à créer un projet simple que vous pouvez utiliser pour compiler une application Compose. Il comprend un seul écran et affiche le texte **"Hello Android!"**.
5. Cliquez sur **Suivant**. La boîte de dialogue **Nouveau projet** s'ouvre. Elle contient des champs permettant de configurer votre projet.
6. Configurez votre projet comme suit :

Le champ **Nom** permet de saisir le nom de votre projet. Pour cet exemple, saisissez "GreetingCard".

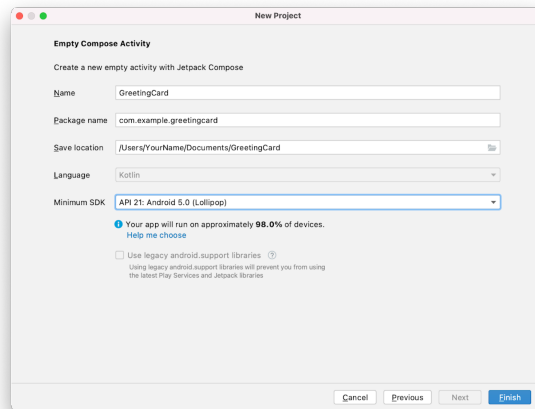
Ne modifiez pas le champ **Nom du package**. C'est de cette façon que vos fichiers seront organisés dans la structure de fichiers. Dans le cas présent, le nom du package sera com.example.greetingcard.

Laissez le champ **Emplacement d'enregistrement** tel quel. C'est là que sont enregistrés tous les fichiers associés à votre projet. Prenez note de cet emplacement sur votre ordinateur pour pouvoir retrouver vos fichiers.

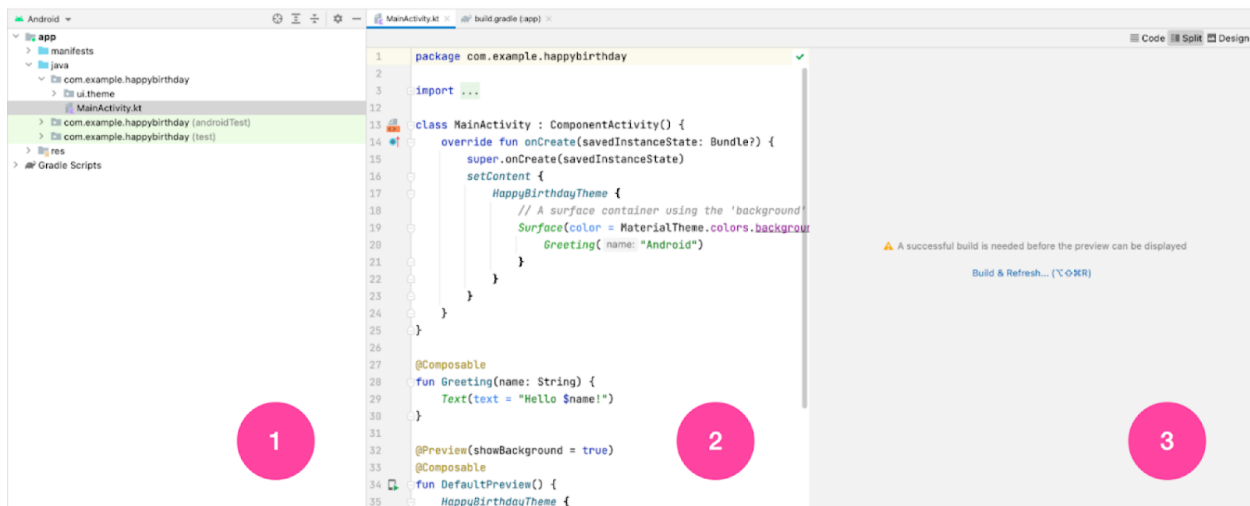
Kotlin est déjà sélectionné dans le champ **Langage**. Il s'agit du langage de programmation que vous souhaitez utiliser pour votre projet. Vous ne pouvez pas modifier ce champ, car Compose n'est compatible qu'avec Kotlin.

Sélectionnez **API 21: Android 5.0 (Lollipop)** dans le menu du champ **SDK minimum**. Ce champ **SDK minimum** indique la version minimale d'Android sur laquelle votre application peut être exécutée.

La case **Utiliser les anciennes bibliothèques android.support** est déjà décochée.



- Appuyez sur **Terminer**. Cette opération peut prendre un certain temps. Pendant la configuration d'Android Studio, une barre de progression et un message indiquent si la configuration du projet est toujours en cours. Un message semblable à celui-ci vous informe lorsque la configuration du projet est terminée: **Gradle sync finished in...**
- Cliquez sur **Diviser** (*Split*) en haut à droite d'Android Studio pour afficher simultanément le code et la conception. Vous pouvez également cliquer sur **Code** pour n'afficher que le code ou sur **Conception** (*Design*) pour n'afficher que la conception.
- Cliquez sur **Compiler et actualiser** (*Build & Refresh*). La compilation peut prendre un certain temps. Une fois qu'elle est terminée, l'aperçu affiche **Hello Android!** dans la zone de texte. Le fichier **MainActivity.kt** contient tout le code nécessaire pour créer cette application.



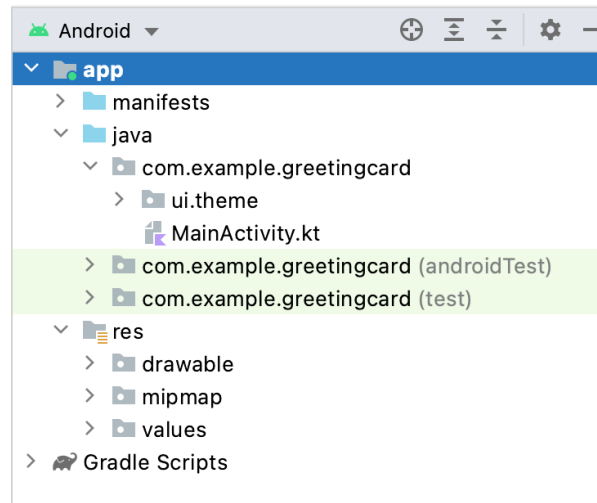
Les fichiers du projet

Dans cette section, vous allez poursuivre l'exploration d'*Android Studio* en vous familiarisant avec la structure de fichiers.

- Dans Android Studio, examinez l'onglet **Projet** (1). L'onglet **Projet** affiche les fichiers et dossiers

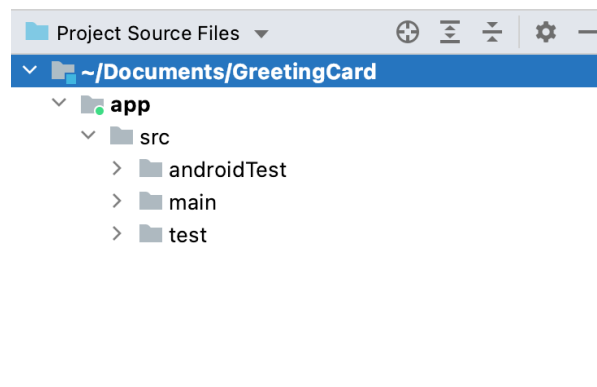
de votre projet. Lorsque vous avez configuré votre projet, le nom du package était **com.example.greetingcard**. Vous pouvez voir ce package, dans l'onglet **Projet**. Un package est en fait un dossier dans lequel se trouve le code. *Android Studio* organise le projet dans une structure de répertoires composée de packages.

2. Si nécessaire, sélectionnez **Android** dans le menu déroulant de l'onglet **Projet**.



il s'agit de l'affichage et de l'organisation standards des fichiers que vous utilisez. Cela s'avère utile lorsque vous écrivez du code pour votre projet, dans la mesure où vous pouvez facilement accéder aux fichiers que vous utiliserez dans votre application. Cependant, si vous consultez les fichiers dans un explorateur de fichiers vous remarquerez que la hiérarchie des fichiers est très différente.

3. Sélectionnez **Fichiers sources du projet** dans le menu déroulant. Vous pouvez maintenant parcourir les fichiers de la même manière que dans n'importe quel autre explorateur de fichiers.



Modification du texte

Maintenant que vous vous êtes familiarisé avec *Android Studio*, il est temps de créer votre carte de présentation.

Examinez la vue **Code** du fichier **MainActivity.kt**. Notez que ce code contient des fonctions

générées automatiquement, et en particulier `onCreate()` et `setContent()`.

```
1 class MainActivity : ComponentActivity() {
2     override fun onCreate(savedInstanceState: Bundle?) {
3         super.onCreate(savedInstanceState)
4         setContent {
5             GreetingCardTheme {
6                 Surface(
7                     modifier = Modifier.fillMaxSize(),
8                     color = MaterialTheme.colors.background
9                 ) {
10                    Greeting("Android")
11                }
12            }
13        }
14    }
15 }
```

La fonction `onCreate()` (ligne 2) est le point d'entrée de cette application et elle appelle d'autres fonctions pour créer l'interface utilisateur. Dans les programmes Kotlin, la fonction `main()` est le point précis où démarre le compilateur Kotlin dans votre code. Dans les applications Android, c'est la fonction `onCreate()` qui remplit ce rôle.

La fonction `setContent()` (ligne 4) permet de définir votre mise en page au moyen de fonctions modulables. Toutes les fonctions annotées avec `@Composable` peuvent être appelées à partir de la fonction `setContent()` ou d'autres fonctions modulables. L'annotation indique au compilateur Kotlin que cette fonction est utilisée par Jetpack Compose pour générer l'interface utilisateur.

Examinez ensuite la fonction `Greeting()`. `Greeting()` est une fonction modulable ; remarquez l'annotation `@Composable` indiquée au-dessus. Une fonction modulable reçoit des entrées et génère ce qui s'affiche à l'écran.

```
1 @Composable
2 fun Greeting(name: String) {
3     Text(text = "Hello $name!")
4 }
```

Vous savez déjà ce que sont les fonctions. Cependant, il existe quelques différences avec les fonctions modulables :

- L'annotation `@Composable` est ajoutée avant la fonction (ligne 1).
- La première lettre d'un nom de fonction `@Composable` est mise en majuscule.
- Les fonctions `@Composable` ne peuvent pas retourner de valeur.

Actuellement, la fonction `Greeting()` accepte un nom et affiche le message "Hello" à la personne dont le nom a été indiqué.

1. Modifiez la fonction `Greeting()` pour vous présenter au lieu de simplement dire "Hello".

```
1 @Composable
2 fun Greeting(name: String) {
3     Text(text = "Bonjour, mon nom est $name")
4 }
```

2. Recréez l'aperçu `DefaultPreview` en appuyant sur **Compiler et actualiser**.

Vous avez modifié le texte, mais il vous présente sous le nom d'Android, qui n'est certainement pas votre véritable identité. Vous allez maintenant le personnaliser pour vous présenter avec votre vrai nom.

`DefaultPreview()` est une fonctionnalité particulièrement utile qui vous permet de voir à quoi ressemble votre application sans avoir à la compiler entièrement. Pour qu'il s'agisse d'une fonction d'aperçu, vous devez ajouter une annotation `@Preview`.

3. Mettez à jour la fonction `DefaultPreview()` avec votre nom. Ensuite, recréez l'aperçu.

```
1 @Preview(showBackground = true)
2 @Composable
3 fun DefaultPreview() {
4     GreetingCardTheme {
5         Greeting("Stephane")
6     }
7 }
```

Modification de la couleur d'arrière-plan

Dans cette section, vous allez apprendre à modifier la couleur de l'arrière-plan.

1. Entourer le composable `Text()` du conteneur `Surface`.

```
1 @Composable
2 fun Greeting(name: String) {
3     Surface() {
4         Text(text = "Bonjour, mon nom est $name")
5     }
6 }
```

2. Le conteneur `Surface` comporte un paramètre `color`. Définissez-le sur `Color`

```

1 @Composable
2 fun Greeting(name: String) {
3     Surface(color = Color) {
4         Text(text = "Bonjour, mon nom est $name")
5     }
6 }

```

3. Lorsque vous saisissez **Color** (Couleur), vous remarquerez peut-être que cet élément est en rouge et qu'il est souligné. Pour résoudre ce problème, faites défiler l'écran jusqu'en haut du fichier, où est indiquée l'instruction **import**, puis appuyez sur les points de suspension.
4. Ajoutez cette instruction en bas de la liste des importations.

```

1 import androidx.compose.ui.graphics.Color

```

5. Choisissez une couleur pour votre surface.

```

1 @Composable
2 fun Greeting(name: String) {
3     Surface(color = Color.Magenta) {
4         Text(text = "Bonjour, mon nom est $name")
5     }
6 }

```

6. Cliquez sur **Compiler et actualiser**.

Ajout de marges intérieures

Maintenant que votre texte a une couleur d'arrière-plan, vous allez ajouter de l'espace (une marge intérieure) autour du texte.

Un élément **Modifier** permet d'améliorer ou de décorer un composable. Vous pouvez utiliser le modificateur **padding**, qui applique un espace autour de l'élément (dans ce cas, il s'agit d'ajouter de l'espace autour du texte). Pour ce faire, utilisez la fonction **Modifier.padding()**.

1. Ajoutez ces importations dans la section des instructions d'importation.

Assurez-vous d'utiliser l'option **Optimiser les importations** pour classer les nouvelles importations par ordre alphabétique.

```

1 import androidx.compose.ui.unit.dp
2 import androidx.compose.foundation.layout.padding

```

2. Ajoutez un modificateur de marge intérieure au texte avec une taille de **24.dp**, puis cliquez sur **Compiler et actualiser**. Les pixels indépendants de la densité (dp) permettent un affichage

uniforme peut importe la résolution et la taille de l'écran d'affichage.

```
1 @Composable
2 fun Greeting(name: String) {
3     Surface(color = Color.Magenta) {
4         Text(text = "Bonjour, mon nom est $name",
5             modifier = Modifier.padding(24.dp))
6     }
7 }
```

Utilisation de l'émulateur Android

Dans cette exemple, vous allez utiliser le **Gestionnaire d'appareils** pour créer un appareil virtuel Android. Un AVD (*Android Virtual Device*) est une version logicielle d'un appareil mobile, également appelée émulateur, qui s'exécute sur votre ordinateur et imite la configuration d'un type d'appareil Android spécifique. Il peut s'agir de n'importe quel téléphone, tablette, TV, montre ou appareil Android Auto.

Créer un AVD

Pour exécuter une application Android dans un émulateur sur votre ordinateur, vous devez d'abord créer un appareil virtuel.

1. Dans Android Studio, sélectionnez **Outils > Gestionnaire d'appareils**.

La boîte de dialogue **Gestionnaire d'appareils** s'ouvre. Si vous avez déjà créé un appareil virtuel, celui-ci est répertorié dans cette boîte de dialogue.

2. Cliquez sur **Create virtual device** (Créer un appareil virtuel).

La fenêtre **Virtual Device Configuration** (Configuration d'un appareil virtuel) s'affiche.

Elle affiche une liste d'appareils préconfigurés, classés par catégorie. Pour chaque appareil, le tableau fournit une colonne pour la taille d'affichage (en pouces), la résolution d'écran (en pixels) et la densité de pixels (nombre de pixels par pouce).

3. Sélectionnez la catégorie **Téléphone**.
4. Sélectionnez un téléphone, comme le Pixel 5, puis cliquez sur **Next** (Suivant).

Cette étape ouvre un autre écran dans lequel vous pouvez choisir la version d'Android à exécuter sur votre appareil virtuel. Vous pouvez ainsi tester votre application sur différentes versions d'Android.

5. Si un lien de téléchargement est affiché à côté de **S**, cliquez sur **Download > Accept > Next > Finish** (Télécharger > Accepter > Suivant > Terminer). La présence du lien de téléchargement indique que l'image n'est pas installée sur votre ordinateur. Dans ce cas, vous devez installer l'image avant de configurer l'appareil virtuel. Le téléchargement peut prendre un certain temps.

6. Dans l'onglet **Recommandé**, sélectionnez **S** comme version d'Android à exécuter sur l'appareil virtuel.

À l'heure où nous écrivons ces lignes, Android S est la dernière version d'Android, mais vous pouvez choisir n'importe quelle version stable ultérieure.

7. Dans le champ **Nom de l'AVD**, saisissez un nom pour votre AVD ou utilisez celui par défaut. Ne modifiez pas les autres champs.
8. Cliquez sur **Terminer**.

Cette action permet de revenir au volet **Gestionnaire d'appareils virtuels Android**.

9. Fermez la fenêtre **Gestionnaire d'appareils**.

Exécuter votre application sur Android Emulator

1. Dans le menu déroulant en haut de la fenêtre Android Studio, sélectionnez l'appareil virtuel que vous avez créé.
2. Cliquez sur >

L'appareil virtuel démarre comme un appareil physique. Le démarrage de l'émulateur peut prendre un certain temps (peut-être plusieurs minutes). L'appareil virtuel devrait s'ouvrir à côté de l'éditeur de code.

Lorsque votre application est prête, elle s'ouvre sur l'appareil virtuel.

3. Mettez à jour la fonction `setContent()` avec votre nom. Ensuite, cliquez sur >.

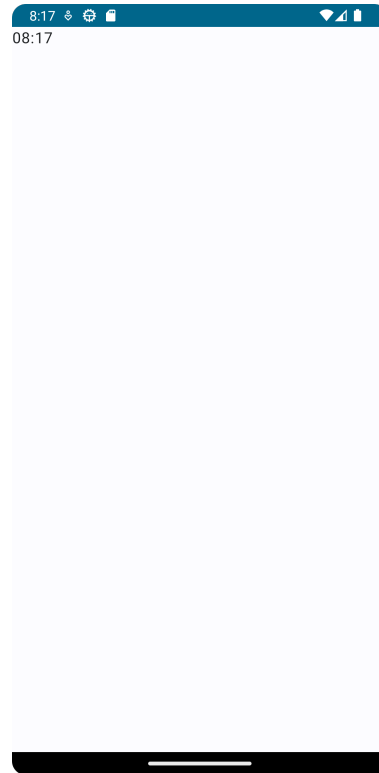
```
1 class MainActivity : AppCompatActivity() {
2     override fun onCreate(savedInstanceState: Bundle?) {
3         super.onCreate(savedInstanceState)
4         setContent {
5             GreetingCardTheme {
6                 Surface(
7                     modifier = Modifier.fillMaxSize(),
8                     color = MaterialTheme.colors.background
9                 ) {
10                    Greeting("Stephane")
11                }
12            }
13        }
14    }
15 }
```



Exercices

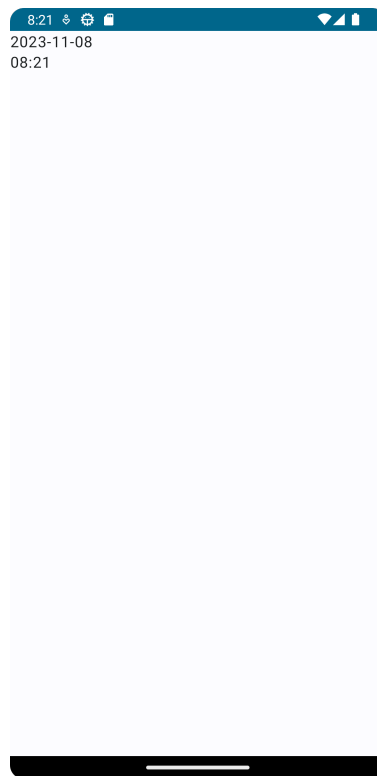
Exercice 1

Faire une application Android qui affiche l'heure.



Exercice 2

Modifier l'application de l'exercice 1 pour ajouter la date d'aujourd'hui.



Références

- Découvrir Android Studio: <https://developer.android.com/studio/intro?hl=fr>
- Principe de base d'android avec Compose: <https://developer.android.com/courses/android-basics-compose/course>
- Texte dans compose: <https://developer.android.com/jetpack/compose/text?hl=fr>